

# NLP Project Report Project

Title: NLP BASED AUTOMATED CLEANSING FOR HEALTHCARE DATA

Date: May 25, 2025

Team Members:

1. Sameeksha: CAN\_33418994
2. Sathvika Jain: CAN\_33416792
3. Swasthika Rai B S: CAN\_33418872
4. Yashika K Y: CAN\_33425899

## 1. Introduction

This project focuses on applying Natural Language Processing (NLP) and Machine Learning (ML) techniques to analyze medical notes with the objective of identifying life-threatening conditions. Unstructured medical text data such as clinical notes, prescriptions, and patient histories are often inconsistent, noisy, and difficult to analyze directly. To address this, the application implements an automated pipeline that preprocesses the raw medical notes through cleaning, lemmatization, and anonymization. It then uses ML models, specifically Random Forest and Support Vector Machine (SVM), to classify the notes based on whether they indicate life-threatening health issues. The system provides an end-to-end solution from data ingestion to analysis, enhancing clinical decision-making capabilities.

## 2. Folder Structure

Project Root/

```
|  
├── backend/  
│   ├── app.py           # Flask server (API endpoints)  
│   ├── healthcare.py    # Core NLP and ML logic  
│   └── uploads/         # Folder to store uploaded CSV files
```

```

|   └── processed/      # Folder where processed CSV files are saved
|
|   └── frontend/
|       ├── public/      # Static files
|       ├── src/         # React frontend source code
|           ├── App.js    # Main React component
|           └── components/ # Modular React components
|
|   ├── medical_records.csv # Input file with medical notes
|   ├── README.md
|   └── .venv/             # Python virtual environment (optional)

```

### 3. Workflow Overview

The application workflow consists of the following stages:

- **CSV Upload:** The user uploads a medical record file via the React.js frontend.
- **Data Preprocessing:** The Flask backend processes the uploaded file through multiple steps:
  - Removal of duplicates.
  - Filling of missing values in the "Notes" column.
  - Text cleaning to remove non-alphanumeric characters.
  - Lemmatization using spaCy to reduce words to their base form.
  - Anonymization of sensitive terms like names and hospital details.
  - Label creation based on the "Life Threats" field.
- **Vectorization:** The preprocessed text is converted to numerical features using TF-IDF.
- **Balancing:** SMOTE (Synthetic Minority Over-sampling Technique) is applied to balance the dataset.
- **Model Training:** Two classifiers, Random Forest and SVM, are trained on the processed data.
- **Evaluation:** Metrics such as precision, recall, F1-score, accuracy, and confusion matrix values are computed and sent back to the frontend.

- **Downloadable Outputs:** Processed files are made available for download in CSV format individually or as a ZIP archive.

## 4. Technologies Used

- **Frontend:**
  - React.js
  - Axios (for API communication)
  - Vercel (for deployment)
- **Backend:**
  - Python 3
  - Flask (web framework)
  - pandas (data manipulation)
  - spaCy (NLP)
  - scikit-learn (machine learning and evaluation metrics)
  - imbalanced-learn (SMOTE balancing)
- **Others:**
  - HTML/CSS for UI styling
  - Git for version control

## 5. Functional Requirements

- **File Upload:** Upload medical records in CSV format.
- **Data Preprocessing:** Clean, lemmatize, and anonymize text.
- **Classification:** Use ML models to predict life-threatening conditions.
- **Results Visualization:** Display model performance metrics.
- **Download Files:** Provide access to download all processed outputs.

## 6. System Requirements

- **Hardware:**
  - Processor: Intel i5 or higher
  - RAM: 8 GB or more
  - Disk Space: Minimum 500 MB free

- **Software:**
  - OS: Windows 10 / Linux / macOS
  - Python 3.7+
  - Node.js and npm (for frontend)
  - React.js
  - Flask
  - Required Python libraries: pandas, scikit-learn, spaCy, imbalanced-learn

## **7. Conclusion**

This project demonstrates how unstructured medical text can be effectively processed and analyzed using NLP and ML techniques. By automating the cleansing and classification of clinical notes, the application provides healthcare professionals with actionable insights that aid in early detection of critical conditions. The modular design and user-friendly interface make it a practical tool for real-world medical data analysis and decision support systems.